Unit 2

Python functions and data structures

Python Functions and Data Structures Function Specifications, Global Variables, Modules, Passing parameters to Functions, Recursive functions, System functions and Parameters, importing modules, Lambda function in python, Python String, List, Tuple, Set, And Dictionary Manipulations, Programming using string, list, tuple, set and dictionary in built functions. **(9)**	**1,2**
File Handling	

FUNCTION

A function is a block of code which only runs when it is called.

You can pass data, known as parameters, into a function.

A function can return data as a result.

defining a function

Here's a simple function named `greet_user()` that prints a greeting:

```
❶ def greet_user():
❷     """Display a simple greeting."""
❸     print("Hello!")

❹ greet_user()
```

This example shows the simplest structure of a function. The line at ❶ uses the keyword `def` to inform Python that you're defining a function. This is the *function definition*, which tells Python the name of the function and, if applicable, what kind of information the function needs to do its job. The parentheses hold that information. In this case, the name of the function is `greet_user()`, and it needs no information to do its job, so its parentheses are empty. (Even so, the parentheses are required.) Finally, the definition ends in a colon.

Passing Information to a Function

```python
def greet_user(username):
    """Display a simple greeting."""
    print("Hello, " + username + "!")
greet_user('python')
```

```
Hello, python!
```

Write a function called favorite_book() that accepts one parameter, title . The function should print a message, such as One of my favorite books is Alice in Wonderland . Call the function, making sure to include a book title as an argument in the function call.

```
[6]  def favorite_book(title):
         """Display a message about a favorite book."""
         print("One of my favorite books is " + title + ".")

     # Function call with a book title as an argument
     favorite_book("Alice in Wonderland")
```

```
One of my favorite books is Alice in Wonderland.
```

Arguments

In the preceding greet_user() function, we defined greet_user() to require a value for the variable username. Once we called the function and gave it the information (a person's name), it printed the right greeting.

An argument is a piece of information that is passed from a function call to a function. When we call the function, we place the value we want the function to work with in parentheses.

Passing arguments

You can use positional arguments, which need to be in the same order the parameters were written; keyword arguments, where each argument consists of a variable name and a value; and lists and dictionaries of values.

Positional Arguments

When you call a function, Python must match each argument in the function call with a parameter in the function definition. The simplest way to do this is based on the order of the arguments provided. Values matched up this way are called positional arguments.

```
❶ def describe_pet(animal_type, pet_name):
    """Display information about a pet."""
    print("\nI have a " + animal_type + ".")
    print("My " + animal_type + "'s name is " + pet_name.title() + ".")

❷ describe_pet('hamster', 'harry')
```

The definition shows that this function needs a type of animal and the animal's name ❶. When we call `describe_pet()`, we need to provide an animal type and a name, in that order. For example, in the function call, the argument `'hamster'` is stored in the parameter `animal_type` and the argument `'harry'` is stored in the parameter `pet_name` ❷. In the function body, these two parameters are used to display information about the pet being described.

The output describes a hamster named Harry:

```
I have a hamster.
My hamster's name is Harry.
```

```python
def describe_pet(animal_type, pet_name):
    """Display information about a pet."""
    print("I have a " + animal_type + ".")
    print("My " + animal_type + "'s name is " + pet_name.title() + ".")
describe_pet('hamster', 'harry')
describe_pet('dog', 'willie')
```

```
I have a hamster.
My hamster's name is Harry.
I have a dog.
My dog's name is Willie.
```

Order Matters in Positional Arguments

You can get unexpected results if you mix up the order of the arguments in a function call when using positional arguments:

```
def describe_pet(animal_type, pet_name):
    """Display information about a pet."""
    print("\nI have a " + animal_type + ".")
    print("My " + animal_type + "'s name is " + pet_name.title() + ".")

describe_pet('harry', 'hamster')
```

In this function call we list the name first and the type of animal second. Because the argument `'harry'` is listed first this time, that value is stored in the parameter `animal_type`. Likewise, `'hamster'` is stored in `pet_name`. Now we have a "harry" named "Hamster":

```
I have a harry.
My harry's name is Hamster.
```

Feature	Positional Arguments	Keyword Arguments
Definition	Arguments passed in the order the parameters are defined.	Arguments passed by explicitly naming the parameter with `=`.
Order	**Order matters** — values are assigned based on position.	**Order does not matter** — values are matched by name.
Readability	Less readable if many parameters are used.	More readable because parameter names are visible in the call.
Example Call	`greet("Alice", 25)`	`greet(age=25, name="Alice")`
Function Definition	Same for both; difference is in how you call it.	Same for both; difference is in how you call it.
When Useful	For short, simple function calls.	When function has many parameters, or to avoid remembering order.

There are three types of functions in Python:

I. Built-in functions The Python interpreter has a number of functions built into it that are always available. They are listed here in alphabetical order.

II. User-Defined Functions (UDFs): The Functions defined by User is known as User Defined Functions. These are defined with the keyword def

III. Anonymous functions, which are also called lambda functions because they are not declared with the standard def keyword.

Built-in Functions				
abs()	divmod()	input()	open()	staticmethod()
all()	enumerate()	int()	ord()	str()
any()	eval()	isinstance()	pow()	sum()
basestring()	execfile()	issubclass()	print()	super()
bin()	file()	iter()	property()	tuple()
bool()	filter()	len()	range()	type()
bytearray()	float()	list()	raw_input()	unichr()
callable()	format()	locals()	reduce()	unicode()
chr()	frozenset()	long()	reload()	vars()
classmethod()	getattr()	map()	repr()	xrange()
cmp()	globals()	max()	reversed()	zip()
compile()	hasattr()	memoryview()	round()	__import__()
complex()	hash()	min()	set()	
delattr()	help()	next()	setattr()	
dict()	hex()	object()	slice()	
dir()	id()	oct()	sorted()	

```
# Python Function with No Arguments, and No Return Value  FUNCTION 1
def Add1():
    a = 20
    b = 30
    Sum = a + b
    print("After Calling the Function:", Sum)
Add1()
```

Here we are not passing any parameter to function instead values are assigned within the function and result is also printed within the function . It is not returning any value

```
# Python Function with Arguments, and No Return Value FUNCTION 2
def Add2(a,b):
    Sum = a + b
        print("Result:", Sum)
Add2(20,30)
```

Here we are passing 2 parameters a,b to the function and function is calculating sum of these parameter and result is printed within the function . It is not returning any value

```
# Python Function with Arguments, and Return Value FUNCTION 3
def Add3(a,b):
    Sum = a + b
    Return Sum

Z=Add3(10,12)
print("Result " Z)
```

Here we are passing 2 parameters a,b to the function and function is calculating sum of these parameter and result is returned to the calling statement which is stored in the variable Z

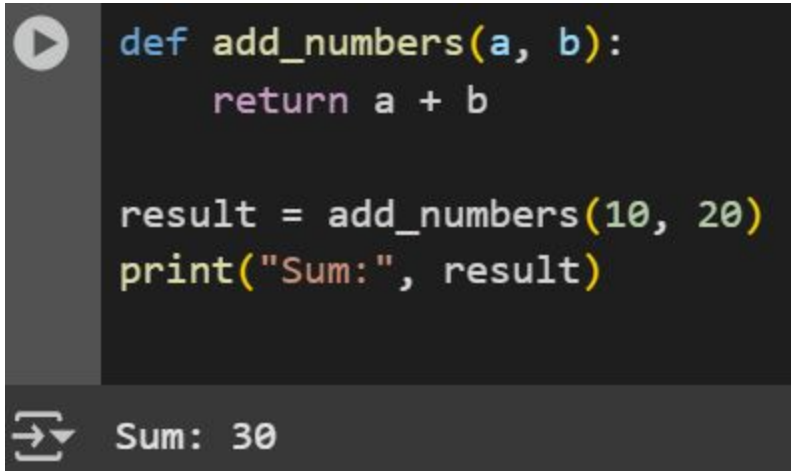
```
# Python Function with No Arguments, and  Return Value FUNCTION 4
def Add4():
    a=11
    b=20
    Sum = a + b
    Return Sum

Z=Add3(10,12)
print("Result " Z)
```

Here we are not passing any parameter to function instead values are assigned within the function but result is returned.

Write a function `add_numbers(a, b)` that returns the sum of two numbers.

```
FUNCTION add_numbers(a, b)
    sum ← a + b
    RETURN sum
END FUNCTION
```

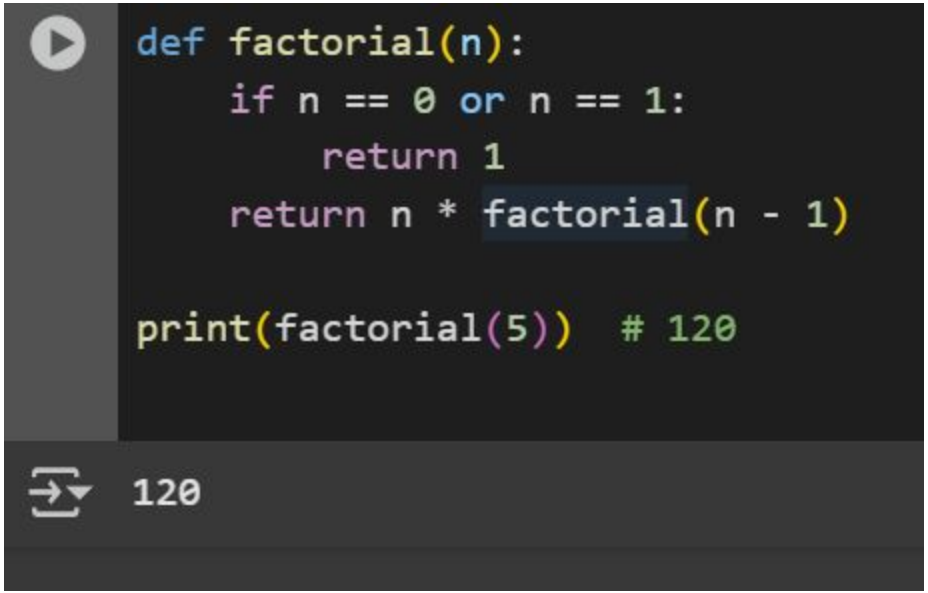

def add_numbers(a, b):
return a + b
result = add_numbers(10, 20)
print("Sum:", result)
Sum: 30

Write a function `factorial(n)` that returns factorial using recursion

Factorial of a number n is defined as:

- **Base case:**
  - `0! = 1`
  - `1! = 1`
- **Recursive case:**
  - `n! = n × (n-1)!`

The function will keep calling itself with `(n-1)` until it reaches `n = 0` or `n = 1`.


def factorial(n):
if n == 0 or n == 1:
return 1
return n * factorial(n - 1)
print(factorial(5)) # 120
120

What is a Lambda Function in Python?

A lambda function is an anonymous function — meaning it has no name.

It’s defined using the keyword lambda and can take any number of arguments but only one expression.

lambda arguments: expression

No def keyword is used.

The expression is automatically returned.

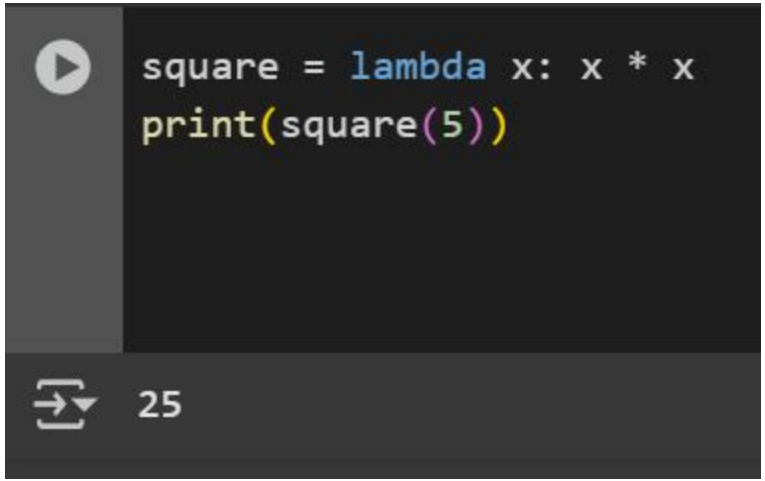
Commonly used for short, throwaway functions.

Write a lambda function to find the square of a number.

Define a lambda function with parameter `x`.

The function evaluates `x * x`.

The result is returned automatically.


square = lambda x: x * x
print(square(5))
25

Cube of a Number

Sum of Two Numbers

Check if a Number is Even or Odd

```python
cube = lambda x: x ** 3
print(cube(4))  # Output: 64
```

```
64
```

```python
sum_two = lambda a, b: a + b
print(sum_two(5, 7))  # Output: 12
```

```
12
```

```python
is_even = lambda x: "Even" if x % 2 == 0 else "Odd"
print(is_even(10))  # Output: Even
print(is_even(7))   # Output: Odd
```

```
Even
Odd
```

Python String

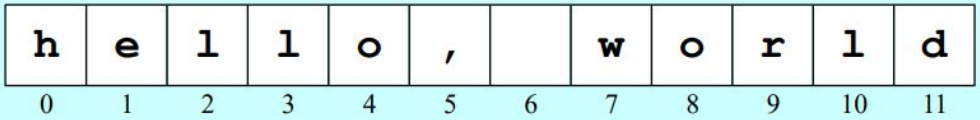
syntax str[k], where k is the index of the desired character.

Selecting Characters from a String

- A string is (still) an ordered collection of characters. The character positions in a Python string are, as in most computer languages, identified by an ***index*** beginning at 0.
- For example, if `s` is initialized as

```
s = "hello, world"
```

the characters in `s` are arranged like this:



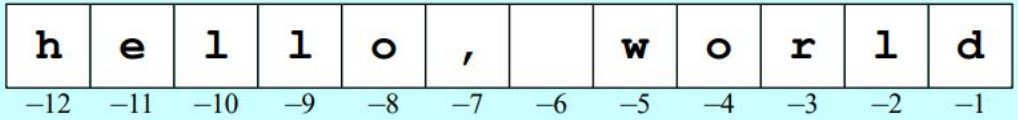
- You can select an individual character using the syntax *str*`[`*k*`]`, where *k* is the index of the desired character. The expression

```
s[7]
```

returns the one-character string `"w"` that appears at index 7.

Negative Indexing

- Unlike JavaScript, Python allows you to specify a character position in a string by using negative index numbers, which count backwards from the end of the string. The characters in the `"hello, world"` string on the previous slide can therefore be numbered using the following indices:



- You can select the `"w"` toward the end of this string using the expression

```
s[-5]
```

which is shorthand for the positive indexing expression

```
s[len(s) - 5]
```

Concatenation

- One of the more familiar operations available to Python strings is ***concatenation***, which consists of combining two strings end to end with no intervening characters.
- Concatenation is built into Python in the form of the + operator. This is consistent with how JavaScript and most other languages support concatenation.
- Noteworthy difference between Python and JavaScript: Python interprets the + operator as concatenation only if **both** operands are strings. If one of the operands is something other than a string, then string concatenation isn't applied. Restated, Python doesn't automatically convert numbers to strings as JavaScript does.

Repetition

- In much the same way that Python redefines the `+` operator to indicate string concatenation, it also redefines the `*` operator for strings to indicate repetition, so that the expression `s * n` indicates `n` copies of the string `s` concatenated together.
- The expression `"la" * 3` therefore returns `"lalala"`, which is three copies of the string `"la"` concatenated together.
- Note that this interpretation is consistent with the idea that multiplication is repeated addition:

```
"la" * 3  →  "la" + "la" + "la"
```

- You can use this feature, for example, to print a line of 80 hyphens like this:

```
print("-" * 80)
```

Slicing

- Python allows you to extract a substring by specifying a range of index positions inside the square brackets. This operation is known as ***slicing***.
- The simplest specification of a slice is `[`*start*`:`*stop*`]`, where *start* is the index at which the slice begins, and *stop* is the past-the-end index where the slice ends.
- The *start* and *stop* components of a slice are optional, but the colon must be present. If *start* is missing, it defaults to 0, and if *stop* is missing, it defaults to the length of the string.
- A slice specification may also contain a third component called a ***stride***, as with `[`*start*`:`*stop*`:`*stride*`]`. Strides indicate how many positions are omitted between selected characters.
- The *stride* component can be negative, in which case the selection occurs backwards from the end of the string.

str`.find(`*pattern*`)`
Returns the first index of *pattern* in *str*, or –1 if it does not appear.

str`.find(`*pattern*`, `*k*`)`
Same as the one-argument version but starts searching from index *k*.

str`.rfind(`*pattern*`)`
Returns the last index of *pattern* in *str*, or –1 if it does not appear.

str`.rfind(`*pattern*`, `*k*`)`
Same as the one-argument version but searches backward from index *k*.

str`.startswith(`*prefix*`)`
Returns `True` if this string starts with *prefix*.

str`.endswith(`*suffix*`)`
Returns `True` if this string ends with *suffix*.

str`.lower()`

Returns a copy of *str* with all letters converted to lowercase.

str`.upper()`

Returns a copy of *str* with all letters converted to uppercase.

str`.capitalize()`

Capitalizes the first character in *str* and converts the rest to lowercase.

str`.strip()`

Removes whitespace characters from both ends of *str*.

str`.replace(`*old*`,` *new*`)`

Returns a copy of *str* with all instances of *old* replaced by *new*.

ch`.isalpha()`
Returns `True` if *ch* is a letter.

ch`.isdigit()`
Returns `True` if *ch* is a digit.

ch`.isalnum()`
Returns `True` if *ch* is a letter or a digit.

ch`.islower()`
Returns `True` if *ch* is a lowercase letter.

ch`.isupper()`
Returns `True` if *ch* is an uppercase letter.

ch`.isspace()`
Returns `True` if *ch* is a ***whitespace character*** (space, tab, or newline).

str`.isidentifier()`
Returns `True` if this string is a legal Python identifier.

1. Length & Accessing

```
s = "Python Programming"

print(len(s))       # Length → 18

print(s[0])         # First character → P

print(s[-1])        # Last character → g

print(s[len(s)//2]) # Middle character → o
```

Slicing

```
s = "PythonIsFun"

print(s[0:6])   # Python

print(s[-3:])   # Fun

print(s[::-1])  # nuFsInohtyP
```

Concatenation

```
a = "Hello"

b = "World"

print(a + " " + b)  # Hello World
```

Negative Start, Stop, and Step Value

We can also use negative numbers for the start, stop, and step arguments. When using positive indices, the first item is 0. When using negative integers, the last item in the sequence is -1.

H	e	l	l	o
0	1	2	3	4
-5	-4	-3	-2	-1

So, if we wanted to start at the last item in the sequence and step down by "1" to "-4", we could specify the start as "-1", the step as "-4", and the step as "-1":

```
sequence = "hello"
new_seq = sequence[-1:-4:-1]
print(new_seq)
```

Write a program that takes a sentence and finds the longest word along with its length.

Print all possible substrings of a given string.

Find the first character in a string that does not repeat anywhere else.

Remove all special characters and digits from a given string, leaving only alphabetic characters.

1. Longest Word in a Sentence

Logic:

1. Split the sentence into words using `split()`.
2. `split()` is a **string method** used to break a string into a **list of substrings**

3. Loop through the words and track the word with the maximum length.

4. Print the longest word and its length.

```python
sentence = input("Enter a sentence: ")
words = sentence.split()

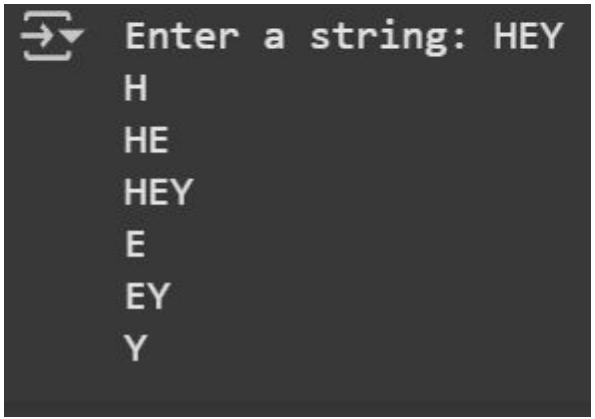
longest_word = max(words, key=len)
print("Longest word:", longest_word)
print("Length:", len(longest_word))
```

```
Enter a sentence: asdf asdfg sdfghj ertyui sdfghjk
Longest word: sdfghjk
Length: 7
```

2. Print All Possible Substrings of a Given String

Logic:

1. Use two nested loops:
    - Outer loop selects the start index.
    - Inner loop selects the end index.
2. Slice the string from `start` to `end` and print it.



```python
text = input("Enter a string: ")

for start in range(len(text)):
    for end in range(start + 1, len(text) + 1):
        print(text[start:end])
```

```
Enter a string: HEY
H
HE
HEY
E
EY
Y
```

3. First Non-Repeating Character in a String

Logic:

1. Loop through the string.
2. For each character, check if its count in the string is 1.
3. Print and break when you find the first such character.

```python
text = input("Enter a string: ")

for char in text:
    if text.count(char) == 1:
        print("First non-repeating character:", char)
        break
else:
    print("No non-repeating character found.")
```

```
Enter a string: eehhjjk
First non-repeating character: k
```

4. Remove All Special Characters and Digits from a String

Logic:

1. Loop through the string.

2. Keep only characters where `isalpha()` is `True`.
3. The **`isalpha()`** method in Python checks **if all characters in a string are alphabetic (A–Z or a–z)**.

4. Join them into a cleaned string.

```python
text = input("Enter a string: ")

cleaned = ''.join(char for char in text if char.isalpha())
print("Cleaned string:", cleaned)
```

```
Enter a string: he,ll./.,'ooo
Cleaned string: hellooo
```

There are four collection data types in the Python programming language:

- List is a collection which is ordered and changeable. Allows duplicate members.
- Tuple is a collection which is ordered and unchangeable. Allows duplicate members.
- Set is a collection which is unordered and unindexed. No duplicate members.
- Dictionary is a collection which is unordered, changeable and indexed. No duplicate members.

When choosing a collection type, it is useful to understand the properties of that type. Choosing the right type for a particular data set could mean retention of meaning, and, it could mean an increase in efficiency or security.

LIST

Lists are used to store multiple items in a single variable.

When we say that lists are ordered, it means that the items have a defined order, and that order will not change.

The list is changeable, meaning that we can change, add, and remove items in a list after it has been created.

List items can be of any data type:

Example

```
list1 = ["apple", "banana", "cherry"]

list2 = [1, 5, 7, 9, 3]

list3 = [True, False, False]
```

A list with strings, integers and boolean values:

```
 list1 = ["abc", 34, True, 40, "male"]
```

Creating Lists

```
numbers = [1, 2, 3, 4, 5]

fruits = ["apple", "banana", "cherry"]

mixed = [1, "hello", True, 3.14]

empty = []
```

Accessing List Items

```
print(fruits[0])    # apple
print(fruits[-1])   # cherry
```

Slicing Lists

```
print(fruits[1:3])  # ['banana', 'cherry']
print(fruits[:2])   # ['apple', 'banana']
```

Modifying List Items

```
fruits[1] = "mango"
print(fruits)  # ['apple', 'mango', 'cherry']
fruits.append("orange")     # Add to end
fruits.insert(1, "grape")   # Insert at position
fruits.remove("banana")     # Remove by value
fruits.pop()                # Remove last item
del fruits[0]               # Delete by index
fruits.clear()              # Empty the list
```

Tuple

A **tuple** is another built-in data structure in Python that is very similar to a list … but **immutable**.

A tuple is an **ordered**, **unchangeable** (immutable) sequence of elements.

Once created, you **cannot** add, remove, or modify its items.

In python, a tuple is a sequence of immutable elements or items.

Tuple is similar to list since the items stored in the list can be changed whereas the tuple is immutable and the items stored in the tuple cannot be changed.

A tuple can be written as the collection of comma-separated values enclosed with the small brackets **( )**.

Syntax: var = **(** value1, value2, value3,.... **)**

Example: "tupledemo.py"

```
t1 = ()
t2 = (123,"python", 3.7)
t3 = (1, 2, 3, 4, 5, 6)
t4 = ("C",)
print(t1)
print(t2)
print(t3)
print(t4)
```

Output:

```
python tupledemo.py
()
(123, 'python', 3.7)
(1, 2, 3, 4, 5, 6)
('C',)
```

+	It is known as concatenation operator used to concatenate two tuples.
*****	It is known as repetition operator. It concatenates the multiple copies of the same tuple.
[]	It is known as slice operator. It is used to access the item from tuple.
[:]	It is known as range slice operator. It is used to access the range of items from tuple.
in	It is known as membership operator. It returns if a particular item is present in the specified tuple.
not in	It is also a membership operator and It returns true if a particular item is not present in the tuple.

Example: "tupleopdemo.py"

```
num=(1,2,3,4,5)
lang=('python','c','java','php')
print(num + lang) #concatenates two tuples
print(num * 2) #concatenates same tuple 2 times
print(lang[2]) # prints 2nd index value
print(lang[1:4]) #prints values from 1st to 3rd index.
print('cpp' in lang) # prints False
print(6 not in num) # prints True
```

Output: **python** tupleopdemo.py

```
(1, 2, 3, 4, 5, 'python', 'c', 'java', 'php')
(1, 2, 3, 4, 5, 1, 2, 3, 4, 5)
java
('c', 'java', 'php')
False
True
```

How to add or remove elements from a tuple?

- Unlike lists, the tuple items cannot be updated or deleted as tuples are immutable.
- To delete an entire tuple, we can use the del keyword with the tuple name.

Example: "tupledemo1.py"

```
tup=('python','c','java','php')
tup[3]="html"
print(tup)
del tup[3]
print(tup)
del tup
```

Output:

```
python tupledemo1.py

'tuple' object does not support item assignment

'tuple' object doesn't support item deletion
```

Why Use Tuples?

- They are **faster** than lists (performance).
- They provide **data protection** (nobody can change them by mistake).
- Suitable for **read-only data**.

- Python provides various in-built functions and methods which can be used with tuples. Those are
  - len()
  - max()
  - min()
  - sum()
  - tuple()
  - sorted()
  - count()
  - index()

☞ len():

- In Python, **len()** function is used to find the length of tuple,i.e it returns the number of items in the tuple.

Syntax: `len(tuple)`

Example: **lendemo.py**

```
num=(1,2,3,4,5,6)
print("length of tuple :",len(num))
```

Output:

```
python lendemo.py
length of tuple : 6
```

Dictionaries

Dictionaries are used to store data values in key: value pairs.

A dictionary is a collection which is ordered, changeable and does not allow duplicates.

Dictionaries are written with curly brackets, and have keys and values

Index values are called keys

Keys are unique within a dictionary while values may not be.

It is generally used when we have a huge amount of data. Dictionaries are optimized for retrieving data. We must know the key to retrieve the value.

Creating a Dictionary

```
student = {
    "name": "John",
    "age": 20,
    "course": "B.Tech"
}
```

- Create and print a dictionary

```
dict = {'Name': 'Zara',
        'Age': 7,
        'Class': 'First'
       }

print(dict)
```

Accessing Values

```
print(student["name"])     # John
print(student.get("age"))  # 20
```

- Create and print a dictionary:

```
dict = { }
dict['Name'] = 'Zara'
dict['Age']=7
dict['Class'] = 'First'
```

Dictionary items are ordered

- When we say that dictionaries are ordered, it means that the items have a defined order, and that order will not change.

Dictionary items are changeable

- Dictionaries are changeable, meaning that we can change, add or remove items after the dictionary has been created.

UPDATE

```
dict = {'Name': 'Zara', 'Age': 7, 'Class': 'First'};

dict['Age'] = 8; # update existing entry
dict['School'] = "DPS School"; # Add new entry


print "dict['Age']: ", dict['Age'];
print "dict['School']: ", dict['School'];
```

When the above code is executed, it produces the following result:

```
dict['Age']:  8
dict['School']:  DPS School
```

```python
# Creating a dictionary
employee = {
    "id": 101,
    "name": "Alice",
    "department": "HR"
}

# Add a new key
employee["salary"] = 45000

# Update value
employee["department"] = "Finance"

# Remove a key
employee.pop("id")

# Display dictionary
for key, value in employee.items():
    print(key, ":", value)
```

```python
# Creating a dictionary
employee = {
    "id": 101,
    "name": "Alice",
    "department": "HR"
}

# Add a new key
employee["salary"] = 45000

# Update value
employee["department"] = "Finance"

# Remove a key
employee.pop("id")

# Display dictionary
for key, value in employee.items():
    print(key, ":", value)
```

```
name : Alice
department : Finance
salary : 45000
```

LIST	TUPLE	DICTIONARY	SET
Allows duplicate members	Allows duplicate members	No duplicate members	No duplicate members
Changeble	Not changeable	Changeable, indexed	Cannot be changed, but can be added, non -indexed
Ordered	Ordered	Unordered	Unordered
Square bracket [ ]	Round brackets ( )	Curly brackets{ }	Curly brackets{ }

t = (15, 7, 22, 7, 10, 7, 5)

How many times does the value 7 occur in the tuple?

What is the **maximum** value in the tuple?

Create a **set** from this tuple and print the set.

Create a **dictionary** that stores each element of the *set* as a key and its **square** as the value.

```
# 1. Count how many times 7 occurs
count_7 = t.count(7)
print("Occurrences of 7:", count_7)
# 2. Maximum value in the tuple
max_val = max(t)
print("Maximum value:", max_val)
```

```
# 3. Create a set from the tuple
s = set(t)
print("Set created from tuple:", s)
# 4. Create dictionary with (element : square)
result_dict = {}
for item in s:
    result_dict[item] = item * item
print("Final dictionary:", result_dict)
```

Occurrences of 7: 3

Maximum value: 22

Set created from tuple: {5, 7, 10, 15, 22}

Final dictionary: {5: 25, 7: 49, 10: 100, 15: 225, 22: 484}